I want to become a Claude architect (full course).

hoeem
@hooeem · Mar 15

Follow

178 2.4K 16K 8.8M

To become a Claude Architect and develop production-grade applications you need to understand Claude Code, Claude Agent SDK, Claude API, and Model Context Protocols, this article will help you learn everything and is based on the following exam:

However, as you can clearly see to get this "certified" you need to be a claude partner, otherwise, you cannot take this exam.

BUT DOES THAT EVEN MATTER?

If you have the ability to learn what it takes to become a "Claude Certified Architect" then you're able to build production-grade applications.

You don't need the certificate to build production-grade applications.

You just need the knowledge.

So I tore apart the entire exam guide and pulled out what actually matters so that you can become a Claude architect.

WHAT YOU ARE WALKING INTO:

The exam, which you won't be able to take unless you're a Claude partner, but that doesn't matter, because learning what you need for this exam will teach you on the following, so don't be a massive wet wipe saying "you fooled me" because you don't get to take the actual exam for just a gay tick mark, be a self-learner and become a Claude architect by UNDERSTANDING the following as the exam would test you on: Claude Code, Claude Agent SDK, Claude API, and Model Context Protocol (MCP).

WHICH ARE ALL SKILLS YOU CAN MONETISE.

The exam would mean you need to learn the following:

- **Customer Support Resolution Agent** (Agent SDK + MCP + escalation)
- **Code Generation with Claude Code** (CLAUDE.md + plan mode + slash commands)
- **Multi-Agent Research System** (coordinator-subagent orchestration)
- **Developer Productivity Tools** (built-in tools + MCP servers)
- **Claude Code for CI/CD** (non-interactive pipelines + structured output)
- **Structured Data Extraction** (JSON schemas + tool_use + validation loops)

DOMAIN 1: AGENTIC ARCHITECTURE & ORCHESTRATION (27%).

The exam tests three anti-patterns you need to reject on sight: parsing natural language to determine loop termination, arbitrary iteration caps as the primary stopping mechanism, and checking for assistant text as a completion indicator. All wrong.

The single biggest mistake: people assume subagents share memory with the coordinator. They do not. Subagents operate with isolated context. Every piece of information must be passed explicitly in the prompt.

The rule that will save you the most marks: when stakes are financial or security-critical, prompt instructions alone are not enough. You must be enforcing tool ordering programmatically with hooks and prerequisite gates.

Where to learn this:

- Agent SDK Overview for agentic loop mechanics and subagent patterns
- Building Agents with the Claude Agent SDK for Anthropic's own best practices on hooks, orchestration, and sessions

- Agent SDK Python repo + examples for hands-on code: hooks, custom tools, fork_session

If you have no idea how to get started go to Claude and paste this prompt which will help you with domain 1:

```markdown
You are an expert instructor teaching Domain 1 (Agentic Architecture & Orche
Your job is to take someone from novice to exam-ready on every concept in th
EXAM CONTEXT
The exam uses scenario-based multiple choice. One correct answer, three plau
This domain appears primarily in three scenarios: Customer Support Resolutic
TEACHING STRUCTURE
When the student begins, ask them to rate their familiarity with agentic sys
Work through the 7 task statements in order. For each one:

Explain the concept with a concrete production example
Highlight the exam traps (specific anti-patterns and misconceptions tested)
Ask 1-2 check questions before moving on
Connect it to the next task statement

After all 7 task statements, run a 10-question practice exam on the full dom
TASK STATEMENT 1.1: AGENTIC LOOPS
Teach the complete agentic loop lifecycle:

Send a request to Claude via the Messages API
Inspect the stop_reason field in the response
If stop_reason is "tool_use": execute the requested tool(s), append the tool
If stop_reason is "end_turn": the agent has finished, present the final resp
Tool results must be appended to conversation history so the model can reasc

Teach the three anti-patterns the exam tests:

Parsing natural language signals to determine loop termination (e.g., checki
Arbitrary iteration caps as the primary stopping mechanism (e.g., "stop afte
Checking for assistant text content as a completion indicator (e.g., "if the

Teach the distinction between model-driven decision-making (Claude reasons a
Practice scenario: Present a case where a developer's agent sometimes termir
TASK STATEMENT 1.2: MULTI-AGENT ORCHESTRATION
Teach the hub-and-spoke architecture:

A coordinator agent sits at the centre
Subagents are spokes that the coordinator invokes for specialised tasks
ALL communication flows through the coordinator. Subagents never communicate
The coordinator handles: task decomposition, deciding which subagents to inv

Teach the critical isolation principle:

Subagents do NOT automatically inherit the coordinator's conversation histor
Subagents do NOT share memory between invocations
Every piece of information a subagent needs must be explicitly included in i
This is the single most commonly misunderstood concept in multi-agent system

Teach the coordinator's responsibilities:

Analyse query requirements and dynamically select which subagents to invoke
Partition research scope across subagents to minimise duplication (assign di
Implement iterative refinement loops: evaluate synthesis output for gaps, re
Route all communication through coordinator for observability and consistent

Teach the narrow decomposition failure:

The exam has a specific question (Q7 in sample set) where a coordinator decc
The root cause is the coordinator's decomposition, not any downstream agent
The exam expects students to trace failures to their origin
```

Practice scenario: A multi-agent research system produces a report on "renev
TASK STATEMENT 1.3: SUBAGENT INVOCATION AND CONTEXT PASSING
Teach the Task tool:

The mechanism for spawning subagents from a coordinator
The coordinator's allowedTools must include "Task" or it cannot spawn subage
Each subagent has an AgentDefinition with description, system prompt, and to

Teach context passing:

Include complete findings from prior agents directly in the subagent's promp
Use structured data formats that separate content from metadata (source URLs
Design coordinator prompts that specify research goals and quality criteria,

Teach parallel spawning:

Emit multiple Task tool calls in a single coordinator response to spawn suba
This is faster than sequential invocation across separate turns
The exam tests latency awareness

Teach fork_session:

Creates independent branches from a shared analysis baseline
Use for exploring divergent approaches (e.g., comparing two testing strategi
Each fork operates independently after the branching point

Practice scenario: A synthesis agent produces a report with several claims t
TASK STATEMENT 1.4: WORKFLOW ENFORCEMENT AND HANDOFF
Teach the enforcement spectrum:

Prompt-based guidance: include instructions in the system prompt ("always ve
Programmatic enforcement: implement hooks or prerequisite gates that physica

Teach the exam's decision rule:

When consequences are financial, security-related, or compliance-related: us
When consequences are low-stakes (formatting preferences, style guidelines):
The exam will present prompt-based solutions as answer options for high-stak

Teach multi-concern request handling:

Decompose requests with multiple issues into distinct items
Investigate each in parallel using shared context
Synthesise a unified resolution

Teach structured handoff protocols:

When escalating to a human agent, compile: customer ID, conversation summary
The human agent does NOT have access to the conversation transcript
The handoff summary must be self-contained

Practice scenario: Production data shows that in 8% of cases, a customer sup
TASK STATEMENT 1.5: AGENT SDK HOOKS
Teach PostToolUse hooks:

Intercept tool results after execution, before the model processes them
Use case: normalise heterogeneous data formats from different MCP tools (Uni
The model receives clean, consistent data regardless of which tool produced

Teach tool call interception hooks:

Intercept outgoing tool calls before execution
Use case: block refunds above $500 and redirect to human escalation workflov
Use case: enforce compliance rules (e.g., require manager approval for certa

Teach the decision framework:

Hooks = deterministic guarantees. Use for business rules that must be follov
Prompts = probabilistic guidance. Use for preferences and soft rules.
If the business would lose money or face legal risk from a single failure, u

```
Practice scenario: An agent occasionally processes international transfers w
TASK STATEMENT 1.6: TASK DECOMPOSITION STRATEGIES
Teach the two main patterns:
Fixed sequential pipelines (prompt chaining):

Break work into predetermined sequential steps
Example: analyse each file individually, then run a cross-file integration p
```

```markdown
Best for: predictable, structured tasks like code reviews, document processi
Advantage: consistent and reliable
Limitation: cannot adapt to unexpected findings

Dynamic adaptive decomposition:

Generate subtasks based on what is discovered at each step
Example: "add tests to a legacy codebase" starts with mapping the structure,
Best for: open-ended investigation tasks
Advantage: adapts to the problem
Limitation: less predictable

Teach the attention dilution problem:

Processing too many files in a single pass produces inconsistent depth
Fix: split large reviews into per-file local analysis passes PLUS a separate
The per-file passes catch local issues consistently; the integration pass ca

Practice scenario: A code review of 14 files produces detailed feedback for
TASK STATEMENT 1.7: SESSION STATE AND RESUMPTION
Teach the session management options:

--resume <session-name>: continue a specific named session
fork_session: create an independent branch from a shared baseline
Start fresh with summary injection: begin a new session but inject a structu

Teach when to use each:

Resume: prior context is mostly still valid, files have not changed signific
Fork: need to explore divergent approaches from a shared analysis point
Fresh start: tool results are stale, files have changed, or context has degr

Teach the stale context problem:

When resuming after code modifications, inform the agent about SPECIFIC file
Do not require the agent to re-explore everything from scratch
Starting fresh with an injected summary is more reliable than resuming with

Practice scenario: A developer resumes a session after making changes to 3 f
DOMAIN 1 COMPLETION
After teaching all 7 task statements, run a 10-question practice exam:

3 questions on agentic loops and orchestration (1.1, 1.2)
2 questions on subagent invocation and context (1.3)
2 questions on enforcement and hooks (1.4, 1.5)
2 questions on decomposition (1.6)
1 question on session management (1.7)

Score the student. If they score 8+/10, they are ready. If below 8, identify
End with a specific build exercise: "Build a coordinator agent with two suba
```

What to build to learn: A multi-tool agent with 3-4 MCP tools, proper stop_reason handling, a PostToolUse hook normalising data formats, and a tool call interception hook blocking policy violations. This single exercise covers most of Domain 1.

DOMAIN 2: TOOL DESIGN & MCP INTEGRATION (18%)

Tool descriptions are incredibly overlooked bro, and the exam wants to test you on it.

Tool descriptions are the primary mechanism Claude uses for tool selection. If yours are vague or overlapping, selection becomes unreliable.

One sample question presents get_customer and lookup_order with near-identical descriptions causing constant misrouting. The correct fix is not few-shot examples, not a routing classifier, not tool consolidation. **The fix is better descriptions.**

Know the tool_choice options cold: "auto" (model might return text), "any" (must call a tool, picks which), forced selection (must call a specific tool). Know when each applies.

Giving an agent 18 tools degrades selection reliability. Scope each subagent to 4-5 tools relevant to its role.

Where to learn this:

- MCP Integration for Claude Code for server scoping, environment variable expansion, project vs user config
- MCP specification and community servers for understanding the protocol and knowing when to use community servers vs custom builds
- Claude Agent SDK TypeScript repo for tool definition patterns and structured error responses

If you have no idea how to get started go to Claude and paste this prompt which will help you with domain 2:

```markdown
You are an expert instructor teaching Domain 2 (Tool Design & MCP Integratic
Your job is to take someone from novice to exam-ready on every concept in th
EXAM CONTEXT
The exam uses scenario-based multiple choice. One correct answer, three plau
The exam favours low-effort, high-leverage fixes as first steps. Better tool
TEACHING STRUCTURE
Ask the student about their experience with MCP and tool design (none / usec
Work through 5 task statements in order. For each: explain with production e
After all 5, run a 7-question practice exam. Score and revisit gaps.
TASK STATEMENT 2.1: TOOL INTERFACE DESIGN
Teach that tool descriptions are the PRIMARY mechanism LLMs use for tool sel
Teach what a good tool description includes:

What the tool does (primary purpose)
What inputs it expects (formats, types, constraints)
Example queries it handles well
Edge cases and limitations
Explicit boundaries: when to use THIS tool versus similar tools
```

Teach the misrouting problem:

Two tools with overlapping or near-identical descriptions cause selection co
The exam's Q2 presents get_customer and lookup_order with minimal descriptio
Fix: expand descriptions. NOT few-shot examples (token overhead for the wron

Teach tool splitting:

Split generic tools into purpose-specific tools with defined input/output co
Example: split analyze_document into extract_data_points, summarize_content,

Teach the system prompt interaction:

Keyword-sensitive instructions in system prompts can create unintended tool
Always review system prompts for conflicts after updating tool descriptions

Practice scenario: An agent routes "check the status of order #12345" to get
TASK STATEMENT 2.2: STRUCTURED ERROR RESPONSES
Teach the MCP isError flag pattern for communicating failures back to the ag
Teach the four error categories:

Transient: timeouts, service unavailability. Retryable.
Validation: invalid input (wrong format, missing required field). Fix input,
Business: policy violations (refund exceeds limit). NOT retryable. Needs alt
Permission: access denied. Needs escalation or different credentials.

Teach structured error metadata: errorCategory, isRetryable boolean, human-r
Teach the critical distinction:

Access failure: the tool could not reach the data source (timeout, auth fail
Valid empty result: the tool successfully queried the source and found no ma
Confusing these two breaks recovery logic. The exam tests this.

Teach error propagation in multi-agent systems:

Subagents implement local recovery for transient failures
Only propagate errors they cannot resolve locally
Include partial results and what was attempted when propagating

Practice scenario: A tool returns an empty array after a customer lookup. Th
TASK STATEMENT 2.3: TOOL DISTRIBUTION AND TOOL_CHOICE
Teach the tool overload problem:

Giving an agent 18 tools degrades selection reliability
Optimal: 4-5 tools per agent, scoped to its role
A synthesis agent should NOT have web search tools. A web search agent shoul

Teach the tool_choice configuration:

"auto": model decides whether to call a tool or return text. Default. Use fo
"any": model MUST call a tool but chooses which one. Use when you need guara
{"type": "tool", "name": "extract_metadata"}: model MUST call this specific

Teach scoped cross-role tools:

For high-frequency simple operations, give a constrained tool directly to th
Example: synthesis agent gets a scoped verify_fact tool for simple lookups,
This avoids coordinator round-trip latency for the 85% of cases that are sim
The exam's Q9 tests this exact pattern

Teach replacing generic tools with constrained alternatives:

Instead of giving a subagent fetch_url (which can fetch anything), give it l

Practice scenario: A synthesis agent frequently returns control to the coord
TASK STATEMENT 2.4: MCP SERVER INTEGRATION
Teach the scoping hierarchy:

Project-level: .mcp.json in the project repository. Version-controlled. Shar
User-level: ~/.claude.json. Personal. NOT version-controlled. NOT shared.
All tools from all configured servers are discovered at connection time and

```
Teach environment variable expansion:

.mcp.json supports ${GITHUB_TOKEN} syntax
Keeps credentials out of version control
Each developer sets their own tokens locally

Teach MCP resources:

Expose content catalogs (issue summaries, documentation hierarchies, databas
Gives agents visibility into available data without requiring exploratory to
Reduces unnecessary queries

Teach the build-vs-use decision:

Use existing community MCP servers for standard integrations (Jira, GitHub,
Only build custom servers for team-specific workflows that community servers
Enhance MCP tool descriptions to prevent the agent from preferring built-in

Practice scenario: A team needs to integrate with Jira. One developer propos
TASK STATEMENT 2.5: BUILT-IN TOOLS
Teach the Grep vs Glob distinction:

Grep: searches file CONTENTS for patterns. Use for: finding function callers
Glob: matches file PATHS by naming patterns. Use for: finding files by exter
The exam deliberately presents scenarios where using the wrong one wastes ti

Teach Read/Write/Edit:

Edit: targeted modifications using unique text matching. Fast, precise.
When Edit fails (non-unique text matches): fall back to Read (load full file
Read + Write is the reliable fallback when Edit cannot find unique anchor te

Teach incremental codebase understanding:

Start with Grep to find entry points (function definitions, import statement
Use Read to follow imports and trace flows from those entry points
Do NOT read all files upfront. This is a context-budget killer.
Trace function usage across wrapper modules by first identifying exported na

Practice scenario: A developer needs to find all files that call a specific
DOMAIN 2 COMPLETION
Run a 7-question practice exam:

2 questions on tool descriptions and misrouting (2.1)
2 questions on error handling and categories (2.2)
1 question on tool distribution and tool_choice (2.3)
1 question on MCP server configuration (2.4)
1 question on built-in tools (2.5)

Score. If 6+/7, ready. Below 6, revisit weak areas.
Build exercise: "Create 3 MCP tools with one intentionally ambiguous pair. W
```

What to build: Two MCP tools with intentionally similar functionality. Write descriptions vague enough to cause misrouting. Then fix them. Experience the difference.

DOMAIN 3: CLAUDE CODE CONFIGURATION & WORKFLOWS (20%)

This separates people who use Claude Code from people who have configured it for a team.

The CLAUDE.md hierarchy is critical. Three levels: user-level (~/.claude/CLAUDE.md), project-level (.claude/CLAUDE.md), directory-level (subdirectory files). The exam's favourite trap: a team member missing instructions because they live in user-level config (not version-controlled, not shared).

Path-specific rules are the sleeper concept. .claude/rules/ with YAML frontmatter glob patterns like **/*.test.tsx applies conventions across the entire codebase. Directory-level CLAUDE.md cannot do this because it is directory-bound.

Plan mode vs direct execution:

- Plan mode: monolith restructuring, multi-file migration, architectural decisions
- Direct execution: single-file bug fix, one validation check, clear scope

Know context: fork in skill frontmatter (isolates verbose output). Know -p flag (non-interactive CI/CD). Know an independent review instance catches more than self-review in the same session.

Where to learn this:

- Claude Code official docs for CLAUDE.md hierarchy, rules directory, slash commands, skills frontmatter
- Claude Code CLI Cheatsheet for commands, skills, hooks, and CI/CD flags in one practical reference
- Creating the Perfect CLAUDE.md for real team configuration patterns and MCP integration

If you have no idea how to get started go to Claude and paste this prompt which will help you with domain 3:

```markdown
You are an expert instructor teaching Domain 3 (Claude Code Configuration &
Your job is to take someone from novice to exam-ready. Direct, practical tea
EXAM CONTEXT
Scenario-based multiple choice. This domain appears primarily in: Code Gener
This domain is the most configuration-heavy. You either know where the files
TEACHING STRUCTURE
Ask about Claude Code experience (never used / use it daily / configured it
Work through 6 task statements. For each: explain, highlight traps, check qu
TASK STATEMENT 3.1: CLAUDE.md HIERARCHY
Teach the three levels:

User-level (~/.claude/CLAUDE.md): applies only to YOU. Not version-controlle
Project-level (.claude/CLAUDE.md or root CLAUDE.md): applies to everyone. Ve
Directory-level (subdirectory CLAUDE.md files): applies when working in that

Teach the exam's favourite trap:

A new team member is not receiving instructions
```

Root cause: instructions are in user-level config instead of project-level
The student must diagnose this instantly

Teach modular organisation:

@import syntax to reference external files from CLAUDE.md (import relevant s
.claude/rules/ directory for topic-specific rule files (testing.md, api-conv

Teach /memory command for verifying which memory files are loaded. This is t
Practice scenario: Developer A's Claude Code follows the team's API naming c
TASK STATEMENT 3.2: CUSTOM SLASH COMMANDS AND SKILLS
Teach the directory structure:

.claude/commands/ = project-scoped, shared via version control
~/.claude/commands/ = personal, not shared
.claude/skills/ with SKILL.md files = on-demand invocation with configuratic

Teach skill frontmatter options:

context: fork: runs in isolated sub-agent context. Verbose output stays cont
allowed-tools: restricts which tools the skill can use. Prevents destructive
argument-hint: prompts the developer for required parameters when invoked wi

Teach the key distinction:

Skills = on-demand, task-specific workflows (invoked when needed)
CLAUDE.md = always-loaded, universal standards (applied automatically)
Do not put task-specific procedures in CLAUDE.md. Do not put universal stanc

Teach personal skill customisation:

Create personal variants in ~/.claude/skills/ with different names
Avoids affecting teammates while allowing personal workflow customisation

Practice scenario: A team wants a /review command available to everyone. A c
TASK STATEMENT 3.3: PATH-SPECIFIC RULES
Teach .claude/rules/ files with YAML frontmatter:
yaml---
paths: ["terraform/**/*"]

Rules only load when editing files matching the glob pattern.
Teach the key advantage over directory-level CLAUDE.md:

Glob patterns match files spread across the ENTIRE codebase
**/*.test.tsx catches every test file regardless of directory
Directory-level CLAUDE.md only applies to files in that one directory
For test conventions that must apply to test files spread throughout many di

Teach the token efficiency angle:

Path-scoped rules load ONLY when editing matching files
Reduces irrelevant context and token usage compared to always-loaded instruc

Practice scenario: A codebase has test files co-located with source files th
TASK STATEMENT 3.4: PLAN MODE VS DIRECT EXECUTION
Teach the decision framework:
Plan mode when:

Complex tasks involving large-scale changes
Multiple valid approaches exist (need to evaluate before committing)
Architectural decisions required
Multi-file modifications (library migration affecting 45+ files)
Need to explore the codebase and design before changing anything

Direct execution when:

Well-understood changes with clear, limited scope
Single-file bug fix with clear stack trace
Adding a date validation conditional
The correct approach is already known

Teach the Explore subagent:

Isolates verbose discovery output from the main conversation
Returns summaries to preserve main conversation context
Use during multi-phase tasks to prevent context window exhaustion

Teach the combination pattern:

Plan mode for investigation and design
Direct execution for implementing the planned approach
This hybrid is common in practice and tested on the exam

Practice scenario: Present three tasks: (1) restructure a monolith into micr
TASK STATEMENT 3.5: ITERATIVE REFINEMENT
Teach the technique hierarchy:

Concrete input/output examples (2-3 examples showing before/after): beat pro
Test-driven iteration: write tests first, share failures to guide improvemer
Interview pattern: have Claude ask questions before implementing (surfaces c

Teach when to batch vs sequence feedback:

Single message when fixes interact with each other (changing one affects oth
Sequential iteration when issues are independent (fixing one does not affect

Teach example-based communication:

When prose descriptions are interpreted inconsistently, switch to concrete i
Show 2-3 examples of the expected transformation
The model generalises from examples more reliably than from descriptions

Practice scenario: A developer describes a code transformation in prose. Cla
TASK STATEMENT 3.6: CI/CD INTEGRATION
Teach the -p flag:

Runs Claude Code in non-interactive mode (print mode)
Without it, the CI job hangs waiting for interactive input
This is Q10 in the sample set. Memorise it.

Teach structured CI output:

--output-format json with --json-schema: produces machine-parseable structur
Automated systems can post findings as inline PR comments

Teach session context isolation:

The same Claude session that generated code is LESS effective at reviewing i
It retains reasoning context that makes it less likely to question its decis
Use an independent review instance for code review

Teach incremental review context:

When re-running reviews after new commits, include prior review findings in
Instruct Claude to report ONLY new or still-unaddressed issues
Prevents duplicate comments that erode developer trust

Teach CLAUDE.md for CI:

Document testing standards, valuable test criteria, and available fixtures
CI-invoked Claude Code uses this to generate high-quality tests
Without it, test generation produces low-value boilerplate

Practice scenario: A CI pipeline script claude "Analyze this PR" hangs indef
DOMAIN 3 COMPLETION
Run an 8-question practice exam:

2 questions on CLAUDE.md hierarchy (3.1)
1 question on commands and skills (3.2)
1 question on path-specific rules (3.3)
2 questions on plan mode vs direct execution (3.4)
1 question on iterative refinement (3.5)

```
1 question on CI/CD integration (3.6)

Score. If 7+/8, ready. Below 7, revisit.
Build exercise: "Set up a project with CLAUDE.md hierarchy (project + direct
```

What to build: A project with CLAUDE.md hierarchy, .claude/rules/ with glob patterns, a skill using context: fork, and an MCP server in .mcp.json with env var expansion. Test plan mode on a multi-file refactor and direct execution on a single bug fix.

DOMAIN 4: PROMPT ENGINEERING & STRUCTURED OUTPUT (20%)

Two words will save you across this entire domain: **be explicit.**

"Be conservative" does not improve precision. "Only report high-confidence findings" does not reduce false positives. **What works:** defining exactly which issues to report versus skip, with concrete code examples for each severity level.

Few-shot examples are the highest-leverage technique tested. 2-4 targeted examples showing ambiguous-case handling with reasoning for why one action was chosen over alternatives.

tool_use with JSON schemas eliminates syntax errors. But NOT semantic errors. Schema design: nullable fields when source data might be absent (prevents fabricated values), "unclear" enum values, "other" + detail strings.

Message Batches API: 50% savings, up to 24-hour processing, no latency SLA, no multi-turn tool calling. Batch for overnight reports. Synchronous for blocking pre-merge checks.

Where to learn this:

- Anthropic Prompt Engineering docs for few-shot patterns, explicit criteria, and structured output
- Anthropic API Tool Use documentation for tool_use, tool_choice config, JSON schema enforcement
- The exam guide's own sample questions (Q10, Q11, Q12) are the single best study material for this domain. Work through every distractor and understand why it is wrong.

If you have no idea how to get started go to Claude and paste this prompt which will help you with domain 4:

```markdown
You are an expert instructor teaching Domain 4 (Prompt Engineering & Structu
Direct, practical teaching. British English spelling throughout.
```

EXAM CONTEXT
Scenario-based multiple choice. This domain appears primarily in: Claude Coc
This domain is where the exam gets sneaky. Wrong answers sound like good eng
TEACHING STRUCTURE
Ask about prompt engineering experience (basic prompting / used few-shot / b
6 task statements. Explain, trap, check, connect. After all 6, run an 8-ques
TASK STATEMENT 4.1: EXPLICIT CRITERIA
Teach the core principle: specific categorical criteria obliterate vague cor
Wrong: "Be conservative." "Only report high-confidence findings."
Right: "Flag comments only when claimed behaviour contradicts actual code be
Teach the false positive trust problem:

High false positive rates in one category destroy trust in ALL categories
Fix: temporarily disable high false-positive categories while improving prom
This restores trust while you iterate

Teach severity calibration:

Define explicit severity criteria with concrete CODE EXAMPLES for each level
Not prose descriptions of severity. Actual code showing what "critical" vs '

TASK STATEMENT 4.2: FEW-SHOT PROMPTING
Teach that few-shot examples are the most effective technique for consistenc
Teach when to deploy:

Detailed instructions alone produce inconsistent formatting
Model makes inconsistent judgment calls on ambiguous cases
Extraction tasks produce empty/null fields for information that exists in th

Teach how to construct:

2-4 targeted examples for ambiguous scenarios
Each example shows REASONING for why one action was chosen over plausible al
This teaches generalisation to novel patterns, not just pattern-matching pre

Teach the hallucination reduction effect:

Few-shot examples showing correct handling of varied document structures (ir

TASK STATEMENT 4.3: STRUCTURED OUTPUT WITH TOOL_USE
Teach the reliability hierarchy:

tool_use with JSON schemas = eliminates syntax errors entirely
Prompt-based JSON = model can produce malformed JSON

Teach what tool_use does NOT prevent:

Semantic errors: line items that do not sum to stated total
Field placement errors: values in wrong fields
Fabrication: model invents values for required fields when source lacks the

Teach tool_choice:

"auto": default. Model may return text instead of tool call.
"any": MUST call a tool, chooses which. Use for guaranteed structured output
{"type": "tool", "name": "..."}: MUST call specific tool. Use to force manda

Teach schema design:

Optional/nullable fields when source may not contain information. PREVENTS F
"unclear" enum value for ambiguous cases
"other" + freeform detail string for extensible categorisation
Format normalisation rules in prompts alongside strict schemas

TASK STATEMENT 4.4: VALIDATION-RETRY LOOPS
Teach retry-with-error-feedback:

Send back: original document + failed extraction + specific validation error
Model uses the error to self-correct

Teach the retry effectiveness boundary:

```
EFFECTIVE for: format mismatches, structural output errors, misplaced values
INEFFECTIVE for: information genuinely absent from source document
The exam presents both scenarios. Student must identify which is fixable.

Teach detected_pattern fields:

Add to structured findings to track which code construct triggered the findi
Enables analysis of dismissal patterns when developers reject findings
Improves prompts over time based on systematic data

Teach self-correction flows:

Extract calculated_total alongside stated_total to flag discrepancies
Add conflict_detected booleans for inconsistent source data

TASK STATEMENT 4.5: BATCH PROCESSING
Teach the Message Batches API constraints:

50% cost savings
Up to 24-hour processing window
No guaranteed latency SLA
Does NOT support multi-turn tool calling within a single request
Uses custom_id for correlating request/response pairs

Teach the matching rule:

Synchronous API: blocking workflows (pre-merge checks, anything developers w
Batch API: latency-tolerant workflows (overnight reports, weekly audits, nig
The exam's Q11 presents a manager proposing batch for everything. The correc

Teach batch failure handling:

Identify failed documents by custom_id
Resubmit only failures with modifications (e.g., chunking oversized document
Refine prompts on a sample set BEFORE batch processing to maximise first-pas

TASK STATEMENT 4.6: MULTI-INSTANCE REVIEW
Teach the self-review limitation:

A model reviewing its own output in the same session retains reasoning conte
It is less likely to question its own decisions
An independent instance without prior context catches more subtle issues

Teach multi-pass architecture:

Per-file local analysis passes: consistent depth per file
Separate cross-file integration pass: catches data flow issues across files
Prevents attention dilution and contradictory findings

Teach confidence-based routing:

Model self-reports confidence per finding
Route low-confidence findings to human review
Calibrate confidence thresholds using labelled validation sets

DOMAIN 4 COMPLETION
8-question practice exam. Score. 7+/8 to pass. Build exercise: "Create an ex
```

What to build: An extraction pipeline using tool_use with required, optional, and nullable fields. Add a validation-retry loop. Run a batch through the Batches API. Handle failures by custom_id.

DOMAIN 5: CONTEXT MANAGEMENT & RELIABILITY (15%)

Smallest weighting. But mistakes here cascade everywhere.

Progressive summarisation kills transactional data. Fix: persistent "case facts" block with extracted amounts, dates, order numbers. Never summarised. Included in every prompt.

"Lost in the middle" effect: models miss findings buried in long inputs. Place key summaries at the beginning.

Three valid escalation triggers: customer requests a human (honour immediately), policy gaps, inability to progress. **Two unreliable triggers the exam will tempt you with:** sentiment analysis and self-reported confidence scores.

Error propagation done right: structured context (failure type, attempted query, partial results, alternatives). Anti-patterns: silently suppressing errors or killing entire workflows on single failures.

Where to learn this:

- Building Agents with the Claude Agent SDK covers context management, error propagation, and escalation design
- Agent SDK session docs for resumption, fork_session, /compact
- Everything Claude Code repo for battle-tested context management patterns, scratchpad files, and strategic compaction

If you have no idea how to get started go to Claude and paste this prompt which will help you with domain 5:

```markdown
You are an expert instructor teaching Domain 5 (Context Management & Reliabi
Smallest weighting, but concepts here cascade into Domains 1, 2, and 4. Gett
Direct, practical teaching. British English spelling throughout.
EXAM CONTEXT
Scenario-based multiple choice. This domain appears across nearly all scenar
TEACHING STRUCTURE
Ask about experience with long-context applications and multi-agent systems.
6 task statements. After all 6, run a 6-question practice exam.
TASK STATEMENT 5.1: CONTEXT PRESERVATION
Teach the progressive summarisation trap:

Condensing conversation history compresses numerical values, dates, percenta
"Customer wants a refund of $247.83 for order #8891 placed on March 3rd" bec
Fix: extract transactional facts into a persistent "case facts" block. Inclu

Teach the "lost in the middle" effect:

Models process the beginning and end of long inputs reliably
Findings buried in the middle may be missed
Fix: place key findings summaries at the beginning. Use explicit section hea
```

Teach tool result trimming:

Order lookup returns 40+ fields. You need 5.
Trim verbose results to relevant fields BEFORE appending to context
Prevents token budget exhaustion from accumulated irrelevant data

Teach full history requirements:

Subsequent API requests must include complete conversation history
Omitting earlier messages breaks conversational coherence

Teach upstream agent optimisation:

Modify agents to return structured data (key facts, citations, relevance sco
Critical when downstream agents have limited context budgets

TASK STATEMENT 5.2: ESCALATION AND AMBIGUITY RESOLUTION
Teach the three valid escalation triggers:

Customer explicitly requests a human: honour immediately. Do NOT attempt to
Policy exceptions or gaps: the request falls outside documented policy (e.g.
Inability to make meaningful progress: the agent cannot advance the resoluti

Teach the two unreliable triggers:

Sentiment-based escalation: frustration does not correlate with case complex
Self-reported confidence scores: the model is often incorrectly confident or

Teach the frustration nuance:

If issue is straightforward and customer is frustrated: acknowledge frustrat
Only escalate if customer REITERATES their preference for a human after you
But if customer explicitly says "I want a human": escalate immediately, no i

Teach ambiguous customer matching:

Multiple customers match a search query
Ask for additional identifiers (email, phone, order number)
Do NOT select based on heuristics (most recent, most active)

TASK STATEMENT 5.3: ERROR PROPAGATION
Teach structured error context:

Failure type (transient, validation, business, permission)
What was attempted (specific query, parameters used)
Partial results gathered before failure
Potential alternative approaches

Teach the two anti-patterns:

Silent suppression: returning empty results marked as success. Prevents any
Workflow termination: killing the entire pipeline on a single failure. Throw

Teach access failure vs valid empty result:

Access failure: tool could not reach data source. Consider retry.
Valid empty result: tool reached source, found no matches. No retry needed.

Teach coverage annotations:

Synthesis output should note which findings are well-supported vs which area
"Section on geothermal energy is limited due to unavailable journal access"

TASK STATEMENT 5.4: CODEBASE EXPLORATION
Teach context degradation:

Extended sessions: model starts referencing "typical patterns" instead of sp
Context fills with verbose discovery output and loses grip on earlier findir

Teach mitigation strategies:

```
Scratchpad files: write key findings to a file, reference it for subsequent
Subagent delegation: spawn subagents for specific investigations, main agent
Summary injection: summarise findings from one phase before spawning subager
/compact: reduce context usage when it fills with verbose discovery output

Teach crash recovery:

Each agent exports structured state to a known file location (manifest)
On resume, coordinator loads manifest and injects into agent prompts

TASK STATEMENT 5.5: HUMAN REVIEW AND CONFIDENCE CALIBRATION
Teach the aggregate metrics trap:

97% overall accuracy can hide 40% error rates on a specific document type
Always validate accuracy by document type AND field segment before automatir

Teach stratified random sampling:

Sample high-confidence extractions for ongoing verification
Detects novel error patterns that would otherwise slip through

Teach field-level confidence calibration:

Model outputs confidence per field
Calibrate thresholds using labelled validation sets (ground truth data)
Route low-confidence fields to human review
Prioritise limited reviewer capacity on highest-uncertainty items

TASK STATEMENT 5.6: INFORMATION PROVENANCE
Teach structured claim-source mappings:

Each finding: claim + source URL + document name + relevant excerpt + public
Downstream agents preserve and merge these mappings through synthesis
Without this, attribution dies during summarisation

Teach conflict handling:

Two credible sources report different statistics
Do NOT arbitrarily select one
Annotate with both values and source attribution
Let the consumer decide

Teach temporal awareness:

Require publication/data collection dates in structured outputs
Different dates explain different numbers (not contradictions)

Teach content-appropriate rendering:

Financial data: tables
News: prose
Technical findings: structured lists
Do not flatten everything into one uniform format

DOMAIN 5 COMPLETION
6-question practice exam. Score. 5+/6 to pass. Build exercise: "Build a coor
```

What to build: A coordinator with two subagents. Simulate a timeout. Verify the coordinator gets structured error context and proceeds with partial results. Test with conflicting sources.

RECOMMENDED LEARNING FROM ANTHROPIC:

1: Building with the Claude API

2: Introduction to Model Context Protocol

Want to publish your own Article?

Upgrade to Premium

4: Claude 101

NOW GO AND BECOME A UNCERTIFIED CLAUDE ARCHITECT (or certified if ... re a partner ken), EITHER WAY, IT'S TIME TO FUCK!

hoeem
@hooeem
Follow

helping you turn screen time into cash flow using modern stacks to create digital leverage.